Project Title

Resume–Job Matching & Talent-Marketplace Engine

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
G. Teja – 2520090162
P. Vedadri Naidu – 2520090060

Team Number: 18

Section: 11

Under the guidance of

Mrs. Ch. Anitha

Asst. Professor, CS&IT



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

The Mini Project Report entitled "**Resume–Job Matching & Talent-Marketplace Engine"** is a record of Bonafide work of **G. Teja – 2520090162, P. Vedadri Naidu – 2520090060** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

G. Teja – 2520090162

P. Vedadri Naidu – 2520090060

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**Resume–Job Matching & Talent-Marketplace Engine"** is a bonafide work done and submitted by **G. Teja – 2520090162, P. Vedadri Naidu – 2520090060** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation to our guide, Mrs. Ch. Anitha, Asst. Professor, CS&IT, which cannot be expressed in words for her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Khaja Shareef**, Head of the *Department of Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of the teaching and non-teaching staff, and all those who have assisted us directly or indirectly, for the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

G. Teja – 2520090162

P. Vedadri Naidu – 2520090060

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	7
4.	Problem Statement and Objectives	9
5.	Hardware and Software requirements	9
6.	Implementation	10
7.	Results & Discussion	13
8.	Conclusion & Future Scope	15
9.	References	16

The Resume–Job Matching & Talent-Marketplace Engine is an intelligent platform that automates the matching of large numbers of résumés to suitable job postings. The system builds an inverted index over candidate résumés and normalizes varied skill names and vocabularies (for example, “JS” and “ECMAScript”, or the typo “pyhton”) into canonical skills using a hash-map synonym table and edit-distance matching. Each candidate is evaluated on skills, qualifications and experience, and a compatibility score between 0 and 100 is computed for every candidate–job pair. A min-heap retrieves the top-K candidates for each job in O(n log K) time, and the Hungarian algorithm finds the one-to-one assignment of candidates to open roles that maximizes total compatibility. A set-cover module then identifies the smallest set of candidates whose combined skills cover every requirement of a team. On a sample dataset of five résumés and three jobs, the engine filled all three roles with a total compatibility of 250 out of 300, and a three-person team covered all eight required skills, matching the exact optimum. By combining résumé indexing, skill normalization, scoring, optimal assignment and skill-set analysis, the engine reduces manual screening effort and supports faster, data-driven hiring decisions.

Team Name: Team 18

Team Logo (if any): Not applicable

Team Members:

1. G. Teja – 2520090162
2. P. Vedadri Naidu – 2520090060

2. INTRODUCTION

Recruitment teams routinely receive hundreds of résumés for every opening. Screening them by hand is slow and inconsistent, and the same skill is written in many different ways ("JS", "JavaScript", "ECMAScript"), so simple keyword search misses good candidates. Ranking each job in isolation has a second weakness: one strong candidate can top the list for several roles while other roles stay unfilled.

This project addresses these problems with a Resume–Job Matching & Talent-Marketplace Engine built from classic data structures and algorithms. Résumés and job requirements are indexed with an inverted index; skill names are normalized with a hash map and edit distance; a weighted compatibility score ranks candidates, with a min-heap used to retrieve the top-K; the Hungarian algorithm assigns candidates to roles optimally; and a set-cover module finds the minimum team that covers all required skills.

The rest of this report is organized as follows: Section 3 surveys related systems, Section 4 states the problem and objectives, Section 5 lists the hardware and software requirements, Section 6 presents the implementation of the four core modules, Section 7 discusses the results, Section 8 concludes with future scope, and Section 9 lists the references.

3. LITERATURE SURVEY

The following existing projects and algorithms were studied. The URLs are given in the References section.

[1] Resume Intelligence Analyzer (FastAPI). A FastAPI service that accepts résumés in several file formats and uses the Google Gemini model to extract text and skills. It assigns each skill a 1–10 score and cross-checks the claimed skills against the candidate's GitHub activity. It scores one candidate at a time and does not decide which candidate should fill which role.

[2] InternMatcher. A Flask and MySQL web application in which a student uploads a PDF résumé, skills are extracted with SpaCy and PyPDF2, and internship listings are ranked by a match score. It serves individual job seekers well, but each user is matched independently of everyone else.

[3] SmartATS. An applicant-tracking tool that parses job descriptions and résumés and ranks candidates with transparent "fit" and "excellence" scores whose weights recruiters can configure per job. A live dashboard shows the ranked candidates. Ranking is done job by job, with no global assignment across jobs.

[4] CommitCV. A hackathon project that ranks a developer's GitHub repositories against a job description using the Gemini API. Its own documentation notes that the ranking logic is basic and could be improved with embeddings, which shows how much accuracy depends on the matching method.

[5] Resume Analyzer for Job Description (MNNIT). A résumé analyzer that extracts skills, experience and education and scores résumés against a job description using TF-IDF similarity, producing technical, managerial and overall match scores. It relies on text similarity and does not normalize skill synonyms explicitly.

[6] Resume ranking course project (text retrieval). A course project that parses job descriptions and résumés and ranks résumés by the skills in the job description, using text-retrieval tools with a customized scoring algorithm and a basic search interface. It focuses on ranking rather than assignment or team composition.

[7] Hungarian algorithm (Kuhn, 1955). A classical method that solves the assignment problem, i.e. pairing n workers with n tasks at minimum total cost, in $O(n^3)$ time. We use it to assign candidates to roles so that total compatibility is maximized.

[8] Set cover problem. A well-known NP-hard problem of choosing the fewest sets whose union covers a given universe. The greedy heuristic gives a good logarithmic approximation, and we use it, verified against an exact search on small inputs, to find the minimum skill set for a team.

Most of the surveyed systems extract skills and rank candidates for a single job or a single user. Our engine goes further by normalizing skill vocabularies, assigning candidates to several roles jointly and optimally, and computing the smallest team that covers all required skills.

4. Problem Statement and Objectives

Problem Statement:

Manual screening of many résumés against many job openings is slow and inconsistent, and it cannot jointly decide which candidate should fill which role.

Problem Description:

Skills are written in many forms and ranking each job separately can give one strong candidate to several roles. The system must normalize skills, score every candidate–job pair, assign candidates optimally, and find the smallest team that covers all required skills.

Objectives:

- Index résumés and job requirements with an inverted index so that only relevant candidates are examined for each job.
- Normalize skill names with a hash-map synonym table and edit distance so that synonyms and typos map to one canonical skill.
- Compute a 0–100 compatibility score from skills (60%), experience (25%) and qualification (15%) for every candidate–job pair.
- Retrieve the top-K candidates for each job efficiently using a min-heap in O(n log K) time.
- Find the optimal one-to-one assignment of candidates to roles that maximizes total compatibility using the Hungarian algorithm.
- Identify the minimum set of candidates whose skills cover all requirements (set cover) and compare the greedy result with the exact optimum.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i3 / AMD Ryzen 3 or above
- RAM: 4 GB minimum (8 GB recommended)
- Storage: 1 GB of free disk space

Software Requirements:

- Operating system: Windows 10/11, Linux or macOS
- Python 3.10 or later (standard library only: heapq, collections, itertools, re)

- Visual Studio Code (editor and terminal)
- Git (optional, for version control)

6. IMPLEMENTATION

The engine is written in Python using only the standard library. The four core modules are given below (file engine.py).

6.1 Module 1: Skill Normalization

Maps each raw skill to a canonical name with an O(1) hash-map lookup and falls back to edit distance (≤ 2) to absorb typos.

```
SYNONYMS = {
    "js": "javascript", "javascript": "javascript", "ecmascript": "javascript",
    "py": "python", "python3": "python", "python": "python",
    "reactjs": "react", "react.js": "react", "react": "react",
    "node": "nodejs", "node.js": "nodejs", "nodejs": "nodejs",
    "ml": "machine learning", "machine learning": "machine learning",
    "postgres": "sql", "mysql": "sql", "sql": "sql", "postgresql": "sql",
    "ds": "data structures", "data structures": "data structures",
    "algos": "algorithms", "algorithms": "algorithms",
    "aws": "cloud", "azure": "cloud", "gcp": "cloud", "cloud": "cloud",
    "rest": "rest api", "rest api": "rest api", "restful api": "rest api",
    "git": "git", "docker": "docker",
}
CANON = sorted(set(SYNONYMS.values()))

def edit_distance(a, b):
    dp = list(range(len(b) + 1))
    for i, ca in enumerate(a, 1):
        prev, dp[0] = dp[0], i
        for j, cb in enumerate(b, 1):
            cur = dp[j]
            dp[j] = min(dp[j] + 1, dp[j - 1] + 1, prev + (ca != cb))
            prev = cur
    return dp[-1]

def normalize_skill(raw):
    s = re.sub(r"\s+", " ", raw.strip().lower())
    if s in SYNONYMS:                       # O(1) hash-map lookup
        return SYNONYMS[s]
    best = min(CANON, key=lambda c: edit_distance(s, c))   # typo tolerance
    return best if edit_distance(s, best) <= 2 else s
```

6.2 Module 2: Inverted Index, Compatibility Score and Top-K

The inverted index returns only résumés that share a skill with the job; a size-K min-heap keeps the best K scores.

```
class Resume:
    def __init__(self, rid, name, skills, years, degree):
        self.rid, self.name, self.years, self.degree = rid, name, years, degree
        self.skills = {normalize_skill(s) for s in skills}

class Job:
    def __init__(self, jid, title, required, min_years, degree):
        self.jid, self.title, self.min_years, self.degree = jid, title, min_years, degree
        self.required = {normalize_skill(s) for s in required}

DEGREE_RANK = {"diploma": 1, "btech": 2, "mtech": 3, "phd": 4}
```

```
class ResumeIndex:
    def __init__(self):
        self.inverted = defaultdict(set)   # skill -> {resume ids}
        self.resumes = {}
    def add(self, r):
        self.resumes[r.rid] = r
        for s in r.skills:
            self.inverted[s].add(r.rid)
    def candidates(self, job):             # only resumes sharing >=1 skill
        out = set()
        for s in job.required:
            out |= self.inverted.get(s, set())
        return out

def score(resume, job):
    skill = len(resume.skills & job.required) / len(job.required)          # Jaccard-style
coverage
    exp = min(resume.years / job.min_years, 1.0) if job.min_years else 1.0
    edu = 1.0 if DEGREE_RANK[resume.degree] >= DEGREE_RANK[job.degree] else 0.5
    return round(100 * (0.6 * skill + 0.25 * exp + 0.15 * edu), 1)

def top_k(index, job, k=3):
    heap = []                              # min-heap of size k -> O(n log k)
    for rid in index.candidates(job):
        sc = score(index.resumes[rid], job)
        if len(heap) < k: heapq.heappush(heap, (sc, rid))
        elif sc > heap[0][0]: heapq.heapreplace(heap, (sc, rid))
    return sorted(heap, reverse=True)
```

6.3 Module 3: Optimal Assignment (Hungarian Algorithm)

Converts scores to costs (100 − score) and finds the one-to-one assignment with the minimum total cost.

```
def hungarian(cost):
    n, m = len(cost), len(cost[0])         # n rows <= m columns
    INF = float("inf")
    u, v, p, way = [0]*(n+1), [0]*(m+1), [0]*(m+1), [0]*(m+1)
    for i in range(1, n+1):
        p[0], j0 = i, 0
        minv, used = [INF]*(m+1), [False]*(m+1)
        while True:
            used[j0] = True
            i0, delta, j1 = p[j0], INF, 0
            for j in range(1, m+1):
                if not used[j]:
                    cur = cost[i0-1][j-1] - u[i0] - v[j]
                    if cur < minv[j]: minv[j], way[j] = cur, j0
                    if minv[j] < delta: delta, j1 = minv[j], j
            for j in range(m+1):
                if used[j]: u[p[j]] += delta; v[j] -= delta
                else: minv[j] -= delta
            j0 = j1
            if p[j0] == 0: break
        while j0:
            j1 = way[j0]; p[j0] = p[j1]; j0 = j1
    return {p[j]-1: j-1 for j in range(1, m+1) if p[j]}   # row -> column

def assign(resumes, jobs):
    cost = [[100 - score(r, j) for r in resumes] for j in jobs]   # maximize score =
minimize cost
    match = hungarian(cost)
```

```
    return [(jobs[i], resumes[c], score(resumes[c], jobs[i])) for i, c in
sorted(match.items())]
```

6.4 Module 4: Minimum Skill Set (Set Cover)

A greedy cover repeatedly picks the résumé covering most uncovered skills; an exact search verifies it on small inputs.

```
def greedy_cover(universe, resumes):
    left, team = set(universe), []
    while left:
        best = max(resumes, key=lambda r: len(r.skills & left))
        if not best.skills & left: break
        team.append(best); left -= best.skills
    return team, left

def optimal_cover(universe, resumes):      # exact, exponential: used to verify greedy on
small inputs
    for size in range(1, len(resumes)+1):
        for combo in combinations(resumes, size):
            if set().union(*(r.skills for r in combo)) >= set(universe):
                return list(combo)
```

7. RESULTS & DISCUSSION

The engine was run on a sample dataset of five résumés (Asha, Ravi, Meena, Kiran, Sneha) and three jobs (Backend, Frontend and ML Engineer). The console output is shown in Fig. 1.

```
== Module 1: skill normalization ==
  JS               -> javascript
  Python3          -> python
  ReactJS          -> react
  pyhton           -> python
  Node.js          -> nodejs
  Postgres         -> sql
  machin learning  -> machine learning
  REST             -> rest api
  Kubernetes       -> kubernetes

== Module 2: index + scoring + top-K (K=3) ==
  J1 Backend Developer: [('Kiran', 70.0), ('Asha', 70.0), ('Meena', 57.5)]
  J2 Frontend Developer: [('Ravi', 100.0), ('Sneha', 72.5)]
  J3 ML Engineer: [('Asha', 80.0), ('Meena', 60.0)]

== Module 3: optimal assignment (Hungarian) ==
  Backend Developer    <- Kiran  (score 70.0)
  Frontend Developer   <- Ravi   (score 100.0)
  ML Engineer          <- Asha   (score 80.0)
  Total compatibility: 250.0

== Module 4: minimum skill set (set cover) ==
  Required skills: ['algorithms', 'docker', 'javascript', 'machine learning', 'python',
'react', 'rest api', 'sql']
  Greedy team  : ['Asha', 'Ravi', 'Meena'] uncovered: []
  Optimal team : ['Asha', 'Ravi', 'Meena']
```

Fig. 1: Console output of the engine on the sample dataset

Module 1 maps synonyms and typos to canonical skills (e.g., “JS” → javascript, “pyhton” → python), while unknown skills such as “Kubernetes” are kept as they are.

Table 1: Top-K candidates per job (compatibility score out of 100)

Job	Rank 1	Rank 2	Rank 3
J1 Backend Developer	Kiran (70.0)	Asha (70.0)	Meena (57.5)
J2 Frontend Developer	Ravi (100.0)	Sneha (72.5)	—
J3 ML Engineer	Asha (80.0)	Meena (60.0)	—

Only résumés sharing at least one required skill are scored, so J2 and J3 list fewer than three candidates. Kiran and Asha tie at 70.0 for J1 and the tie is broken by résumé id.

Table 2: Optimal assignment (Hungarian algorithm)

Job	Assigned candidate	Score
Backend Developer	Kiran	70.0
Frontend Developer	Ravi	100.0
ML Engineer	Asha	80.0
Total		250.0 / 300

All three roles are filled by different candidates with a total compatibility of 250 out of a possible 300.

For the minimum skill set, the eight skills required across the three jobs are covered by a team of three (Asha, Ravi, Meena); the greedy result is identical to the exact optimum on this input.

Table 3: Data structure / algorithm and time complexity of each module

Module	Technique	Time complexity
Skill normalization	Hash map + edit distance	O(1) lookup; $O(\|C\| \cdot L^2)$ fuzzy fallback
Indexing	Inverted index (skill → résumés)	O(total skills) to build
Top-K ranking	Min-heap of size K	O(n log K)
Assignment	Hungarian algorithm	$O(n^3)$
Minimum skill set	Greedy set cover (verified by exact search)	Greedy $O(\|U\| \cdot m)$; exact is exponential

The results come from a small sample dataset; they demonstrate that each module behaves correctly, but they are not a benchmark on large real-world data.

8. CONCLUSION & FUTURE SCOPE

All the objectives were met. The engine indexes résumés, normalizes skill vocabularies, scores every candidate–job pair, retrieves the top-K candidates with a min-heap, assigns candidates to roles optimally with the Hungarian algorithm, and finds a minimum team covering all required skills. On the sample data it filled every role with a total compatibility of 250 out of 300 and covered all eight skills with three candidates, equal to the exact optimum. The work shows that classic data structures and algorithms provide a fast and explainable basis for automated hiring decisions.

Future scope

- **Near term:** add a web interface (React front end with a backend API and database) and automatic résumé parsing from PDF/DOCX files.
- **Medium term:** learn the score weights from past hiring data, use semantic embeddings for skill matching, and allow one candidate to hold more than one role or a role to need several people.
- **Long term:** evaluate on large real-world datasets, add fairness and bias checks, and support continuous matching as new résumés and jobs arrive.

9. REFERENCES

[1] Resume Intelligence Analyzer (FastAPI), GitHub. https://github.com/TulikaZeth/GutHib_FastAPI

[2] InternMatcher – Resume-Based Job Matcher, GitHub. https://github.com/Ashmita67/InternMatcher-Enhanced

[3] SmartATS – Automated Applicant Scoring & Ranking, GitHub. https://github.com/saber-030/SmartATS

[4] CommitCV, FOSS Hack 2025. https://fossunited.org/hack/fosshack25/p/9jher9qm5l

[5] Resume Analyzer for Job Description, MNNIT SAC. https://sac.mnnit.ac.in/robotics/projects/overview/74

[6] Resume ranking course project (text retrieval), GitHub. https://github.com/dkrovi2/CourseProject/pull/1/files

[7] Hungarian algorithm, Wikipedia. https://en.wikipedia.org/wiki/Hungarian_algorithm

[8] Set cover problem, Wikipedia. https://en.wikipedia.org/wiki/Set_cover_problem

[9] H. W. Kuhn, "The Hungarian method for the assignment problem," Naval Research Logistics Quarterly, vol. 2, no. 1–2, pp. 83–97, 1955.

[10] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.